

ĐẠI HỌC BÁCH KHOA HÀ NỘI
TRƯỜNG CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG



BLOCKCHAIN AND APPLICATIONS
CAPSTONE PROJECT REPORT

TOPIC: NFT MARKETPLACE WITH FRACTIONAL ROYALTIES

Course Instructor: Mr. Nguyễn Bình Minh

Group: 3

Members:

Chu Văn An – 20235581

Lê Duy Anh – 20225565

Trần Chí Hiếu - 20225572

Class ID: 168076

Hà Nội, tháng 6 năm 2026

Table of Contents

1. Introduction
2. Project Overview and Task Assignment
3. Technical Requirement 1 — Advanced ERC-721 & EIP-2981
4. Technical Requirement 2 — Payment Splitter (Pull-Payment Pattern)
5. Technical Requirement 3 — Atomic Swaps in Marketplace
6. Contract Interaction Flow
7. Step-by-Step Testing Guide
8. Security Considerations
9. Conclusion
10. References

Github repository link: <https://github.com/Chuan30/nft-fractional-marketplace>

1. Introduction

This report documents the design and implementation of a Fractional Royalty NFT Marketplace — a set of smart contracts built on Ethereum that allow artists and creators to mint NFTs, set up multi-receiver royalty splits, and trade them securely in a decentralized marketplace.

The project was developed as part of a Blockchain Development course and focuses on three core technical requirements: (1) extending the ERC-721 standard with EIP-2981 multi-receiver royalties, (2) implementing a pull-payment splitter that is safe against reentrancy and out-of-gas errors, and (3) building an atomic swap marketplace that transfers NFT and distributes funds in one transaction.

All contracts are written in Solidity ^0.8.20 using OpenZeppelin libraries and are designed to be deployed and tested via Remix IDE. The code is intentionally kept simple and readable so that other students can understand and build upon it.

2. Project Overview and Task Assignment

2.1 Project Structure

The project consists of three Solidity source files, each responsible for a distinct layer of the system:

File	Contract(s)	Role
Token.sol	FractionalRoyaltyNFT, RoyaltySplitter	Minting NFTs + attaching fractional royalty splitters
PaymentSplitter.sol	PaymentSplitter	Core pull-payment logic for dividing ETH by shares
Marketplace.sol	AtomicNFTMarketplace	Listing & buying NFTs with atomic royalty distribution

2.2 How They Fit Together

When an artist mints an NFT via FractionalRoyaltyNFT, they specify a list of payees (team members, collaborators, etc.) and their respective shares. The contract deploys a dedicated RoyaltySplitter (which extends PaymentSplitter) for that specific token and sets it as the EIP-2981 royalty receiver. When the NFT is sold on the AtomicNFTMarketplace, the buyer pays ETH, the marketplace computes the royalty via EIP-2981, sends the royalty amount to the splitter, sends the rest to the seller, and transfers the NFT — all in one atomic transaction. Each royalty stakeholder can then call `release()` on the splitter to claim their share at any time.

2.3 Task Assignment

Chu Văn An – 20235581: Group Leader, write and test the main branch, do the slides.

Lê Duy Anh – 20225565: Write and test the backup (Remix IDE) version, do the demo and presentation.

Trần Chí Hiếu – 20225572: Create a Remix IDE version of the main branch, write the report.

3. Technical Requirement 1 — Advanced ERC-721 & EIP-2981

3.1 What is EIP-2981?

EIP-2981 is the Ethereum standard for NFT royalties. It defines a function `royaltyInfo(tokenId, salePrice)` that returns a receiver address and the royalty amount. Any marketplace that supports EIP-2981 can query this function and automatically send royalties to the right place — without needing to know the contract's internal logic.

3.2 Our Implementation: Multi-Receiver Fractional Royalties

Normal EIP-2981 only supports one royalty receiver per token. Our project expands on this by allowing the creator to specify an array of payees and their share percentages. Here is how it works:

- When `mintWithFractionalRoyalty()` is called, the contract:
- Creates a new `RoyaltySplitter` contract with the given payees and shares.
- Maps the token ID to this splitter address in the `tokenSplitters` mapping.
- Calls `_setTokenRoyalty(tokenId, splitterAddress, feeNumerator)` so that EIP-2981 points to the splitter.

Now when a marketplace calls `royaltyInfo()`, it gets back the splitter contract address. The marketplace sends the royalty ETH to the splitter, and each payee can later call `release()` to withdraw their portion.

```
function mintWithFractionalRoyalty(
    address to,
    address[] memory payees,
    uint256[] memory shares,
    uint96 feeNumerator
) public onlyOwner returns (uint256) {
    uint256 tokenId = _nextTokenId++;
    _safeMint(to, tokenId);

    RoyaltySplitter splitter = new RoyaltySplitter(payees, shares);
    tokenSplitters[tokenId] = address(splitter);
    _setTokenRoyalty(tokenId, address(splitter), feeNumerator);

    return tokenId;
}
```

3.3 Why This Matters

This approach keeps royalty distribution decentralized and transparent. Each NFT has its own isolated splitter, so different NFTs can have completely different royalty structures. The

marketplace does not need to know about the splitter — it just follows EIP-2981, and the royalties end up in the right hands.

4. Technical Requirement 2 — Payment Splitter (Pull-Payment Pattern)

4.1 The Problem: Sending ETH to Many People

If a contract tries to 'push' ETH to 10 royalty receivers in a single transaction, several things can go wrong:

- Reentrancy attack — a malicious receiver's fallback function calls back into the contract.
- Out-of-gas error — sending to 10 addresses costs a lot of gas; if it runs out mid-loop, some people get paid and some don't.
- One receiver reverts — if one receiver's fallback rejects ETH, the whole transaction fails and nobody gets paid.

4.2 The Solution: Pull-Payment (Claim-Based) Pattern

Instead of pushing ETH to everyone, we use a pull-payment pattern. The contract keeps track of how much each payee is entitled to, and each payee calls `release()` to withdraw their funds when they want. This is the same pattern used by OpenZeppelin's `PaymentSplitter`.

4.3 How Our `PaymentSplitter` Works

- Constructor — takes an array of payees and an array of shares. Validates that no address is zero, shares are positive, and there are no duplicates. Stores everything in mappings.
- `receive()` — the contract can accept ETH natively (no function needed).
- `release(address payable account)` — calculates how much ETH the account is owed: $(\text{totalReceived} * \text{accountShares} / \text{totalShares}) - \text{alreadyReleased}$. If the result is positive, it updates the released amount and sends the ETH via `Address.sendValue()`.

```
function release(address payable account) public virtual {
    require(_shares[account] > 0, "Account has no shares");

    uint256 totalReceived = address(this).balance + _totalReleased;
    uint256 payment = (totalReceived * _shares[account])
        / _totalShares - _released[account];
    require(payment > 0, "Account is not due payment");

    _released[account] += payment;
    _totalReleased += payment;

    account.sendValue(payment);
}
```

4.4 Security Analysis

- Reentrancy — the balance is updated (effects) before the ETH is sent (interaction). This follows the Checks-Effects-Interactions pattern. Even if the receiver re-enters `release()`, the math will show they have already been paid.
- Out-of-gas — each `release()` only sends to one address at a time. There is no loop over payees. Each person claims individually, so gas cost per transaction is small and predictable.
- One bad receiver — if a receiver's address rejects ETH (e.g. a contract with no `receive()`), only that person's transaction fails. Everyone else can still claim their funds.

5. Technical Requirement 3 — Atomic Swaps in Marketplace

5.1 What is an Atomic Swap?

An atomic swap means that either everything happens or nothing happens. In our marketplace, when someone buys an NFT, three things must happen together:

- The NFT is transferred from seller to buyer.
- The royalty amount is sent to the royalty splitter.
- The remaining payment goes to the seller.

If any of these fail, the entire transaction reverts — the buyer gets their money back, the seller keeps their NFT, and nothing is left in an inconsistent state.

5.2 Implementation in AtomicNFTMarketplace

The buyNFT function uses OpenZeppelin's nonReentrant modifier and follows the Checks-Effects-Interactions pattern:

- Check — verifies the listing is active and funds are sufficient.
- Effects — marks the listing as inactive (isActive = false). This prevents reentrancy.
- Interactions — (a) query EIP-2981 royaltyInfo, (b) send royalty to splitter via .call{value}(), (c) send seller proceeds via .call{value}(), (d) transfer NFT via safeTransferFrom().
- If the buyer overpays, the excess is refunded at the end — also as part of the same atomic transaction.

```
function buyNFT(address nftAddress, uint256 tokenId)
    external payable nonReentrant {
    Listing storage listing = listings[nftAddress][tokenId];
    require(listing.isActive, "Listing not active");
    require(msg.value >= listing.price, "Insufficient funds");

    // Effects
    listing.isActive = false;

    // Query EIP-2981 Royalty
    if (IERC165(nftAddress).supportsInterface(
        type(IERC2981).interfaceId)) {
        (receiver, amount) = IERC2981(nftAddress)
            .royaltyInfo(tokenId, salePrice);
    }

    // Interactions: send ETH, then transfer NFT
    if (receiver != address(0) && amount > 0) {
        receiver.call{value: amount}("");
    }
    seller.call{value: sellerProceeds}("");
```

```
IERC721(nftAddress).safeTransferFrom(  
    seller, msg.sender, tokenId);  
}
```

5.3 Why SafeTransferFrom?

safeTransferFrom calls the receiver's onERC721Received() to ensure the receiver is capable of handling ERC-721 tokens. If the target is an exchange or another contract that does not support ERC-721, the transfer reverts — preventing tokens from being locked in incompatible contracts forever.

6. Contract Interaction Flow

Below is a step-by-step walkthrough of how the three contracts interact during the complete lifecycle of an NFT — from minting to royalty claiming.

Step 1 — Mint (Artist)

The artist calls `mintWithFractionalRoyalty()` on the `FractionalRoyaltyNFT` contract. They provide the recipient address, an array of payees (people who should get royalties), an array of share percentages, and a fee numerator (e.g. $500 = 5\%$). The contract mints a new ERC-721 token and deploys a fresh `RoyaltySplitter` contract with the given payees and shares. It stores the splitter address in the `tokenSplitters` mapping and calls `_setTokenRoyalty()` to register the splitter as the EIP-2981 royalty receiver for that token.

Step 2 — Approve Marketplace (Seller)

Before listing, the seller must call `setApprovalForAll()` on the NFT contract, passing the marketplace address and `true`. This gives the marketplace permission to transfer the NFT when a sale happens. Without this step, the marketplace cannot move the NFT and the buy transaction will revert.

Step 3 — List NFT (Seller)

The seller calls `listNFT()` on the `AtomicNFTMarketplace` contract with the NFT contract address, token ID, and a price in wei. The marketplace stores a `Listing` struct with the seller's address, the NFT details, and sets `isActive` to `true`. An `NFTListed` event is emitted.

Step 4 — Buy NFT (Buyer)

The buyer calls `buyNFT()` on the marketplace, sending ETH along with the transaction. The amount sent must be at least the listing price. The marketplace then executes these sub-steps atomically (if any fails, the whole transaction reverts):

- Checks that the listing is active and the buyer sent enough ETH.
- Marks the listing as inactive (`isActive = false`) to prevent reentrancy.
- Queries the NFT contract via EIP-165 to check if it supports EIP-2981. If it does, it calls `royaltyInfo(tokenId, salePrice)` to get the royalty receiver and amount.
- Sends the royalty amount to the receiver (which is the `RoyaltySplitter` contract).
- Sends the remaining sale proceeds (`salePrice - royalty`) to the seller.
- Calls `safeTransferFrom()` to move the NFT from the seller to the buyer.
- If the buyer sent more ETH than the price, the excess is refunded.

Step 5 — Claim Royalties (Each Payee)

After the sale, the `RoyaltySplitter` contract holds the royalty ETH. Each payee can independently call `release()` on the splitter contract, passing their own address. The splitter calculates how

much ETH they are owed based on their share of the total, deducts any amount already claimed, and sends the ETH via `sendValue()`. This pull-payment pattern means there is no loop over payees — each person claims individually, so there is no risk of out-of-gas errors or one receiver blocking others.

Summary of the Full Flow

Artist mints an NFT with a custom royalty splitter attached to it. Seller approves the marketplace and lists the NFT. Buyer purchases the NFT in one atomic transaction — royalty goes to the splitter, proceeds go to the seller, and the NFT goes to the buyer. Finally, each royalty stakeholder claims their share from the splitter at any time.

7. Step-by-Step Testing Guide

All testing can be done inside Remix IDE using the Remix VM environment. No external tools or testnets are required.

Step 1: Deploy FractionalRoyaltyNFT

Select FractionalRoyaltyNFT from the dropdown, paste your address as initialOwner, and click Deploy.

Step 2: Deploy AtomicNFTMarketplace

Select AtomicNFTMarketplace and click Deploy. Note both deployed addresses.

Step 3: Mint an NFT

Call mintWithFractionalRoyalty with:

- to: your address
- payees: e.g. ["0x...B", "0x...C"]
- shares: e.g. [70, 30]
- feeNumerator: e.g. 500 (5% royalty)

Step 4: Approve the Marketplace

Call setApprovalForAll on the NFT contract:

- operator: marketplace address
- approved: true

Step 5: List the NFT

Call listNFT on the marketplace:

- nftAddress: NFT contract address
- tokenId: 0
- price: e.g. 1 ether

Step 6: Buy the NFT (switch accounts)

Change to a different account in Remix. Call buyNFT with:

- nftAddress: NFT contract address
- tokenId: 0
- Value field: 1 ether (or more to test refunds)

Step 7: Verify balances

Check that the seller received ETH, the splitter has royalty funds, and the buyer now owns the NFT.

Step 8: Claim royalties

Find the splitter address via `tokenSplitters(0)`. Switch to the splitter contract. Each payee calls `release(theirAddress)` to withdraw.

7.1 Testing with Different Scenarios

Try edge cases like: buying with exact ETH vs overpaying, having a payee who never claims, or trying to buy an already-sold listing (should revert). All of these help verify that the atomic swap and pull-payment patterns work correctly.

8. Security Considerations

Reentrancy Protection

The marketplace uses OpenZeppelin's ReentrancyGuard (nonReentrant modifier) and the splitter uses Checks-Effects-Interactions. Both contracts update state before making external calls.

Pull-Payment over Push

Instead of looping over payees in a distribution function, we let each payee pull their own funds. This eliminates out-of-gas risks and prevents one bad receiver from blocking others.

Safe Transfers

safeTransferFrom ensures that the receiving contract understands ERC-721. This prevents NFTs from being permanently stuck in contracts that don't support them.

EIP-2981 Introspection

The marketplace checks supportsInterface() before calling royaltyInfo(). If the NFT contract does not support EIP-2981, no royalty is sent — the seller gets the full amount.

Excess Refund

If the buyer sends more ETH than the asking price, the difference is refunded. This happens inside the same atomic transaction, so there is no risk of funds getting stuck.

No Floating Code

The splitter is deployed per-token, not reused. Each token has an isolated, non-upgradeable splitter, so a compromised token cannot affect others.

9. Conclusion

This project demonstrates how to build a real-world NFT system using Solidity and OpenZeppelin while following Ethereum best practices. The three technical requirements — EIP-2981 multi-receiver royalties, pull-payment splitting, and atomic swaps — are implemented in a clean, modular way that is easy to understand and test.

The Fractional Royalty NFT Marketplace shows that complex features like fractional royalties and secure fund distribution can be achieved by composing simple, well-tested building blocks. Each contract has a single responsibility: `FractionalRoyaltyNFT` handles minting and royalty configuration, `PaymentSplitter` handles fund distribution, and `AtomicNFTMarketplace` handles trading with atomic execution guarantees.

This architecture makes the system easy to extend. New features such as auctions, offer systems, or batched minting could be added without changing the core contracts. The use of EIP standards (ERC-721, EIP-2981) ensures compatibility with the broader Ethereum ecosystem, including marketplaces, wallets, and indexing services.

10. References

- OpenZeppelin Contracts — <https://github.com/OpenZeppelin/openzeppelin-contracts>
- EIP-721: Non-Fungible Token Standard — <https://eips.ethereum.org/EIPS/eip-721>
- EIP-2981: NFT Royalty Standard — <https://eips.ethereum.org/EIPS/eip-2981>
- EIP-165: Standard Interface Detection — <https://eips.ethereum.org/EIPS/eip-165>
- Solidity Documentation — <https://docs.soliditylang.org/>
- Remix IDE — <https://remix.ethereum.org/>
- Checks-Effects-Interactions Pattern — <https://docs.soliditylang.org/en/latest/security-considerations.html>